- No Results Found for "crossover"

HTTP API

Welcome to Arylic's developer documentation.
This documentation describes the API of DIY audio boards, which are manufactured by the Linkplay company. Their boards are so called white label solutions, which means that hundreds of brands use them in their own products (as Arylic does). There are many well-known manufacturers that use Linkplay as their core. Theoretically, most of them should be controllable with the API described below.

Abstract

Assuming a device IP address of `192.168.10.1`, the general request format would look like:

```
GET http://192.168.10.1/httpapi.asp?command={command}
```

Arylic's `Axx` modules are SoC modules for WiFi Audio solutions, which supports Smartlink, DLNA, Spotify Connect and Airplay. They also provide support for an `HTTP` API to get quick access.

To communicate with a board you have to send `HTTP GET` requests.
The response is in `JSON` format.

> All response examples in this documentation are from different devices (**Up2Stream Amp v4**, **Up2Stream mini v3**) but same firmware version (`4.6.327252.26`). It is possible that responses from older firmware may differ.

Networking

List of scanned AP's

Request format:

```
GET /httpapi.asp?command=wlanGetApListEx
```

Example response:

```
{
  "res": "2",
  "aplist": [
    {
      "auth": "AES",
      "bssid": "b4:fb:e4:f4:73:81",
      "channel": "11",
      "extch": "0",
      "rssi": "37",
```

```
        "ssid": "57696E6B656C6761737365"
      },
      {
        "auth": "WPA2PSK",
        "bssid": "18:e8:29:9d:75:c5",
        "channel": "6",
        "encry": "AES",
        "extch": "0",
        "rssi": "24",
        "ssid": "426574656967657757A65556E696669"
      }
    ]
}
```


Retrieves a list of nearby scanned WiFi Access Points and reports some of the main properties. The JSON table results will be sorted by signal strength `RSSI`

Command: `wlanGetApListEx`

DESCRIPTION OF RESPONSE VALUES

Key	Value Description
`res`	Number of SSID's found
`aplist`	The key to get the list of scanned Access Points

DESCRIPTION OF AN AP RESPONSE OBJECT

Key	Value Description
`auth`	Required WiFi authorization mechanism
`bssid`	The MAC address of that WiFi
`channel`	Used WiFi channel
`extch`	*!! DOCUMENTATION IN PROGRESS !!*
`rssi`	RSSI (Received Signal Strength Indication) value Value range is from `0 - 100`. `100` means best signal strength.
`ssid`	The SSID of that WiFi network `[hexed string]`

Connect to WiFi

Request format:

```
GET /httpapi.asp?command=wlanConnectApEx:
    ssid=<hex_ssid>:ch=<num_channel>:auth=<text_auth>:encry=<text_encry>:pwd=<hex_pwd>:chext=1
```

Request to connect to a desired WiFi network. Mainly used during device setup.

Command: `wlanConnectApEx`

Parameter	Description
`hex_ssid`	`[hexed string]` of the WiFi SSID
`num_channel`	WiFi channel to connect (1-12)
`text_auth`	WiFi authorization mechanism, `OPEN` or `WPA2PSK`
`text_encry`	Encryption type, `AES` or `NONE`
`hex_pwd`	`[hexed string]` of the WiFi password
`flag_chext`	1

This request has no response, you can call **`wlanGetConnectState`** to get the connection status.

Connect to hidden WiFi

Request format with password:

```
GET /httpapi.asp?command=wlanConnectHideApEx:ssid=<hex_ssid>:pwd=<hex_pwd>
```

Request format without password:

```
GET /httpapi.asp?command=wlanConnectHideApEx:ssid=<hex_ssid>
```

Connect the device to a hidden WiFi network. When connecting, the device connection may be lost.

Command: `wlanConnectHideApEx`

Parameter	Description
`hex_ssid`	`[hexed string]` of the WiFi SSID
`hex_pwd`	`[hexed string]` of the WiFi password (optional value)

This request has no response, you can call **'wlanGetConnectState'** to get the connection state.

Device connection status

Request format:

```
GET /httpapi.asp?command=wlanGetConnectState
```

Example response:

```
OK
```

Command: `wlanGetConnectState`

This command will return the status of the WiFi connection. The possible return values are as follows.

Value	Value Description
`PROCESS`	Connection still in progress.
`PAIRFAIL`	WiFi connection attempt failed. Wrong password given.
`FAIL`	WiFi connection attempt failed. Also this will be the reply for a device that is connected by LAN Ethernet port.
`OK`	Device is connected.

ℹ The response value is NOT in JSON format! It is just a plaintext string.

Query networking status

Request format:

```
GET /httpapi.asp?command=getStaticIP
```

Example response for ETH DHCP:

```
{
    "wifi": "-1",
    "eth": "1"
}
```

Example response for ETH Manual IP:

```
{
    "wifi": "-1",
    "eth": "0",
    "eth_static_ip": "192.168.10.103",
    "eth_static_mask": "255.255.255.0",
    "eth_static_gateway": "192.168.10.1",
    "eth_static_dns1": "192.168.10.1",
    "eth_static_dns2": ""
}
```

Query current networking status

Command: `getStaticIP`

DESCRIPTION OF RESPONSE VALUES

Key	Value Description
`wifi`	state of WiFi: `-1` - not connected

Key	Value Description
	`0` - Static IP `1` - DHCP
`eth`	state of ETH: `-1` - not connected `0` - Static IP `1` - DHCP
`eth_static_ip`	Available when `eth` is `0`, Manually set IP address
`eth_static_mask`	Available when `eth` is `0`, Net mask
`eth_static_gateway`	Available when `eth` is `0`, Gateway
`eth_static_dns1`	Available when `eth` is `0`, Prefered DNS
`eth_static_dsn2`	Available when `eth` is `0`, Alternate DNS
`wifi_static_ip`	Available when `wifi` is `0`, Manually set IP address
`wifi_static_mask`	Available when `wifi` is `0`, Net mask
`wifi_static_gateway`	Available when `wifi` is `0`, Gateway
`wifi_static_dns1`	Available when `wifi` is `0`, Prefered DNS
`wifi_static_dsn2`	Available when `wifi` is `0`, Alternate DNS

Introduced in version 4.6.328252

Set Manual IP

Request format:

```
GET /httpapi.asp?command=setStaticIP:{"type":"<TYPE>","ip":"<IP>","mask":"<MASK>","gateway":"<GW>","dns":[{"service":"<DNS1>"},{"service":"<DNS2>"}]}
```

Sample Response:

```
OK
```

Manually set network parameters for WiFi or Ethernet. The API will accept a json format string as parameter.

Command: `setStaticIP:<info>`

PARAMETERS IN `INFO`:

Field	Description
`TYPE`	Network interface to set, could be `wifi` or `eth`
`IP`	IPV4 address to manually set, eg: `192.168.0.100`

Field	Description
`MASK`	network mask, eg: `255.255.255.0`
`GW`	Gateway, eg: `192.168.0.1`
`DNS1`	Prefered DNS, eg: `8.8.8.8`
`DNS2`	Alternate DNS, eg: `8.8.8.8`

Introduced in version 4.6.328252

Set DHCP

Request format:

```
GET /httpapi.asp?command=setDhcp:<TYPE>
```

Sample Response:

```
OK
```

Request to connect to a desired WiFi network. Mainly used during device setup.

Command: `setDhcp`

Command parameters:

Parameter	Description
`TYPE`	Network interface to set, could be `wifi` or `eth`

Introduced in version 4.6.328252

Device Information

Get Device Metadata

Request format:

```
GET /httpapi.asp?command=getStatusEx
```

Example response:

```
{
   "uuid": "FF31F09E1A5020113B0A3918",
```

```
"DeviceName": "PRO",
"GroupName": "PRO",
"ssid": "PRO_D4AD",
"language": "en_us",
"firmware": "4.6.328252",
"hardware": "A31",
"build": "release",
"project": "UP2STREAM_PRO_V3",
"priv_prj": "UP2STREAM_PRO_V3",
"project_build_name": "a31rakoit",
"Release": "20210903",
"temp_uuid": "A7A50887ACBC9B36",
"hideSSID": "1",
"SSIDStrategy": "2",
"branch": "A31_stable_4.6",
"group": "0",
"wmrm_version": "4.2",
"internet": "1",
"MAC": "00:22:6C:33:D4:AD",
"STA_MAC": "00:00:00:00:00:00",
"CountryCode": "CN",
"CountryRegion": "1",
"netstat": "0",
"essid": "",
"apcli0": "",
"eth2": "192.168.167.74",
"ra0": "10.10.10.254",
"eth_dhcp": "0",
"eth_static_ip": "192.168.167.74",
"eth_static_mask": "255.255.255.0",
"eth_static_gateway": "192.168.167.1",
"eth_static_dns1": "194.168.4.100",
"eth_static_dns2": "194.168.8.100",
"VersionUpdate": "0",
"NewVer": "0",
"mcu_ver": "28",
"mcu_ver_new": "0",
"dsp_ver": "",
"dsp_ver_new": "0",
"date": "2021:09:27",
"time": "13:25:17",
"tz": "0.0000",
"dst_enable": "1",
"region": "unknown",
"prompt_status": "1",
"iot_ver": "1.0.0",
"upnp_version": "1005",
"cap1": "0x305200",
"capability": "0x28e90b80",
"languages": "0x6",
"streams_all": "0x7bff7ffe",
"streams": "0x7b9831fe",
"external": "0x0",
"plm_support": "0x6",
"preset_key": "10",
"spotify_active": "1",
"lbc_support": "0",
"privacy_mode": "0",
"WifiChannel": "11",
"RSSI": "0",
"BSSID": "",
"battery": "0",
"battery_percent": "0",
"securemode": "1",
"auth": "WPAPSKWPA2PSK",
"encry": "AES",
"upnp_uuid": "uuid:FF31F09E-1A50-2011-3B0A-3918FF31F09E",
"uart_pass_port": "8899",
"communication_port": "8819",
"web_firmware_update_hide": "0",
"ignore_talkstart": "0",
"web_login_result": "-1",
```

```
    "silenceOTATime": "",
    "ignore_silenceOTATime": "1",
    "new_tunein_preset_and_alarm": "1",
    "iheartradio_new": "1",
    "new_iheart_podcast": "1",
    "tidal_version": "2.0",
    "service_version": "1.0",
    "security": "https\/2.0",
    "security_version": "2.0"
}
```


Retrieves detailed informations about a connected device.

Command: `getStatusEx`

DESCRIPTION OF RESPONSE VALUES

Key	Value Description
`uuid`	Device permanent UUID (will remain after device reboot)
`DeviceName`	The device UPnP and Airplay friendly name
`GroupName`	*!! DOCUMENTATION IN PROGRESS !!*
`ssid`	Device's own `SSID` when in WiFi pairing mode or when device's WiFi hotspot is active
`language`	The language
`firmware`	Current firmware version
`hardware`	Hardware model
`build`	Possible values: `release`, `debug`, `backup` `release`: this is a release version `debug`: this is a debug version `backup`: this is a backup version
`project`	The project name
`priv_prj`	Project name which would stand for a specific board
`project_build_name`	Code identifier for customized release
`Release`	Firmware build date Format: `YYYYMMDD`
`temp_uuid`	Temporary UUID (will change after device reboot)
`hideSSID`	When the device is operating as a WiFi hotspot, this flag determines whether its SSID should be hidden or visible `0`: `ssid` is visible `1`: `ssid` is hidden
`SSIDStrategy`	*!! DOCUMENTATION IN PROGRESS !!*
`branch`	Code branch
`group`	Wether the device is working slave mode, `0` means master or standalone.
`master_uuid`	Exist when working in slave mode, showing the `UUID` of master device.

Key	Value Description
`slave_mask`	Exist when working in slave mode, showing if the device support mask feature. `0` means not supported.
`wmrm_version`	Multiroom library version, the latest version 4.2 is not compatible with 2.0.
`internet`	Current status of internet access: `0`: not ready `1`: ready
`MAC`	MAC address of the device when working in hotspot mode, will show on APP and also the sticker on module/device.
`STA_MAC`	MAC address of the `STA` = STATION
`CountryCode`	*!! DOCUMENTATION IN PROGRESS !!*
`CountryRegion`	*!! DOCUMENTATION IN PROGRESS !!*
`netstat`	WiFi connect state: `0`: no connection `1`: connecting `2`: connected
`essid`	SSID of the WiFi the device is connected to `[hexed string]`
`apcli0`	Device's IP address over WiFi
`eth2`	Device's IP address when it's connected to ethernet
`ra0`	WiFi AP IP address, normally it is `10.10.10.254`
`eth_dhcp`	Flag for DHCP or Static IP Address `0`: Static IP `1`: IP Address provided by LAN/WLAN DHCP Server
`eth_static_ip`	Device's Static IP address (If `eth_dhcp=0`)
`eth_static_mask`	Device's Network Mask (If `eth_dhcp=0`)
`eth_static_gateway`	Device's IP Gateway (If `eth_dhcp=0`)
`eth_static_dns1`	Device's Primary DNS Server (If `eth_dhcp=0`)
`eth_static_dns2`	Device's Secondary DNS Server (If `eth_dhcp=0`)
`VersionUpdate`	Flag that determines, if there is a new firmware version available or not. `0`: no new firmware `1`: new firmware available
`NewVer`	If there is a new firmware available (in case of `VersionUpdate` is set to `1`), this is the new version number
`mcu_ver`	Version of MCU on base board
`mcu_ver_new`	New version of MCU on base board, indicates if there is a newer version of MCU available `0` - No new version `others` - New version
`dsp_ver`	Version for voice processing, not used

Key	Value Description
`dsp_ver_new`	New version for voice processing, not used
`date`	Current Date
`time`	Current local time
`tz`	Offset of timezone
`dst_enable`	*!! DOCUMENTATION IN PROGRESS !!*
`region`	*!! DOCUMENTATION IN PROGRESS !!*
`prompt_status`	Indicates if the prompting voice would be played or not, you can set with command `PromptEnable` and `PromptDisable`. `0` - No prompting voice `1` - Have prompting voice
`iot_ver`	IOT library version, not used
`upnp_version`	UPnP Device Architecture Version
`cap1`	Bit mask for the module feature, used internally
`capability`	Bit mask for the module feature, used internally
`languages`	*!! DOCUMENTATION IN PROGRESS !!*
`streams_all`	*!! DOCUMENTATION IN PROGRESS !!*
`streams`	This is a bit mask: `0`: If Airplay is enabled `1`: If DLNA is enabled `2`: Has TTPod support `3`: Has TuneIn support `4`: Has Pandora support `5`: Has DoubanFM support !! DOCUMENTATION IN PROGRESS !!*
`external`	hexadecimal value *!! DOCUMENTATION IN PROGRESS !!*
`plm_support`	This is a bit mask, each bit stands for an external input source: `bit1`: LineIn (Aux support) `bit2`: Bluetooth support `bit3`: USB support `bit4`: Optical support `bit6`: Coaxial support `bit8`: LineIn 2 support `bit15`: USBDAC support Others are reserved or not used.
`preset_key`	Quantity of presets available:
`spotify_active`	*!! DOCUMENTATION IN PROGRESS !!* But I guess: Flag that indicates if Spotify is currently playing on the device (via Spotify Connect?) `0`: Spotify is not playing `1`: Spotify is playing
`lbc_support`	*!! DOCUMENTATION IN PROGRESS !!*

Key	Value Description
`privacy_mode`	*!! DOCUMENTATION IN PROGRESS !!*
`WifiChannel`	The current connected WiFi channel
`RSSI`	RSSI Level of used WiFi Value ranges from `0 - 100`. `100` means best signal strength.
`BSSID`	The Basic Service Set Identifiers : In most cases this will be the MAC Address of the Wireless Acces Point Used (e.g. Router)
`battery`	`0`: battery is not charging `1`: battery is charging
`battery_percent`	Battery charge level Value ranges from `0 - 100`
`securemode`	*!! DOCUMENTATION IN PROGRESS !!*
`auth`	Type of WiFi Protected Access used (Authentication Key).
`encry`	Type of WiFi Protected Access used (Encryption Protocol).
`upnp_uuid`	The UPnP UUID
`uart_pass_port`	Port used for TCP/IP Communcations/Socket Connections
`communication_port`	TCP port for internal messages
`web_firmware_update_hide`	*!! DOCUMENTATION IN PROGRESS !!*
`ignore_talkstart`	*!! DOCUMENTATION IN PROGRESS !!*
`silenceOTATime`	*!! DOCUMENTATION IN PROGRESS !!*
`ignore_silenceOTATime`	*!! DOCUMENTATION IN PROGRESS !!*
`new_tunein_preset_and_alarm`	*!! DOCUMENTATION IN PROGRESS !!*
`iheartradio_new`	*!! DOCUMENTATION IN PROGRESS !!*
`new_iheart_podcast`	*!! DOCUMENTATION IN PROGRESS !!*
`tidal_version`	TIDAL API version
`service_version`	*!! DOCUMENTATION IN PROGRESS !!*
`security`	*!! DOCUMENTATION IN PROGRESS !!*
`security_version`	*!! DOCUMENTATION IN PROGRESS !!*

There are **LOTS** of keys in the example response which don't have any description yet!

Get System Log

Request format:

```
GET /httpapi.asp?command=getsyslog
```

Example response:

```
<!doctype html>
<html>
    <head>
        <meta charset="utf-8">
    </head>
    <body>
        <DIV>
            <span id="dl">    <a href=data/sys.log>download</a><hr></span>
        </DIV>
    </body>
</html>
```

To get current system log for debugging, the content is encryted. The response for this command is special, it will show a download link for user. And after clicked, will download the log file onto your computer.

Command: `getsyslog`

Redirect OTA Server

Request format:

```
GET /httpapi.asp?command=SetUpdateServer:<url>
```

Example response:

```
OK
```

It is used to redirect the OTA server, so user can select different OTA server manually, normally for alpha testing or pre-release purpose. Afer redirected, the web page will response with OK, and user need to check the APP for the device and see if there's a new version. And click to update when found new version. After next boot, the OTA server will resume to default one. And another point, the MCU firmware will be detected as new version when found different, even it's larger.

Command: `SetUpdateServer:<url>`

PARAMETER DESCRIPTION

Parameter	Description
`url`	URL of OTA server, and for Arylic device, you can set it to `http://ota.rakoit.com/alpha`

Reboot Device

Request format:

```
GET /httpapi.asp?command=reboot
```

It is used to reboot the device, in case of some condition that the device might be unstable after working for a very long time.

Command: `reboot`

Playback Control

Playback Status

Command: `getPlayerStatus`

Request format:

```
GET /httpapi.asp?command=getPlayerStatus
```

Example response:

```
{
    "type":"0",
    "ch":"0",
    "mode":"10",
    "loop":"3",
    "eq":"0",
    "status":"play",
    "curpos":"11",
    "offset_pts":"11",
    "totlen":"170653",
    "Title":"596F752661706F733B766520476F7420746865204C6F7665205B2A5D",
    "Artist":"466C6F72656E636520616E6420746865204D616368696E65",
    "Album":"4C756E6773205B31362F34345D",
    "alarmflag":"0",
    "plicount":"11",
    "plicurr":"9",
    "vol":"47",
    "mute":"0
}
```

DESCRIPTION OF RESPONSE VALUES

Key	Value Description
`type`	`0`: Main or standalone device `1`: Device is a Multiroom Guest
`ch`	Active channel(s) `0`: Stereo `1`: Left `2`: Right
`mode`	Playback mode `0`: Idling `1`: airplay streaming `2`: DLNA streaming `10`: Playing network content, e.g. vTuner, Home Media Share, Amazon Music, Deezer, etc. `11`: playing UDISK(Local USB disk on Arylic Device) `20`: playback start by HTTPAPI `31`: Spotify Connect streaming `40`: Line-In input mode

Key	Value Description
	`41`: Bluetooth input mode `43`: Optical input mode `47`: Line-In #2 input mode `51`: USBDAC input mode `99`: The Device is a Guest in a Multiroom Zone
`loop`	Is a Combination of `SHUFFLE` and `REPEAT` modes `0`: `SHUFFLE:` disabled `REPEAT:` enabled - loop `1`: `SHUFFLE:` disabled `REPEAT:` enabled - loop once `2`: `SHUFFLE:` enabled `REPEAT:` enabled - loop `3`: `SHUFFLE:` enabled `REPEAT:` disabled `4`: `SHUFFLE:` disabled `REPEAT:` disabled `5`: `SHUFFLE:` enabled `REPEAT:` enabled - loop once
`eq`	The current Equalizer setting
`status`	Device status `stop`: no audio selected `play`: playing audio `load`: load **??** `pause`: audio paused
`curpos`	Current playing position (in ms)
`offset_pts`	*!! DOCUMENTATION IN PROGRESS !!*
`totlen`	Current track length (in ms)
`Title`	`[hexed string]` of the track title
`Artist`	`[hexed string]` of the artist
`Album`	`[hexed string]` of the album
`alarmflag`	*!! DOCUMENTATION IN PROGRESS !!*
`plicount`	The total number of tracks in the playlist
`plicurr`	Index of current track in playlist
`vol`	Current volume Value range is from `0 - 100`. So can be considered a linear percentage (`0% to 100%`)
`mute`	The mute status `0`: Not muted `1`: Muted

Select Input Source

Request format:

```
GET /httpapi.asp?command=setPlayerCmd:switchmode:<player_mode>
```

Example Response:

```
OK
```

Selects the Audio Source of the Device. The available audio sources for each device will depend on the installed hardware.

Command: `setPlayerCmd:switchmode`

Parameter	Description
`player_mode`	The audio source that has to be switched `wifi`: wifi mode `line-in`: line analogue input `bluetooth`: bluetooth mode `optical`: optical digital input `co-axial`: co-axial digital input `line-in2`: line analogue input #2 `udisk`: UDisk mode `PCUSB`: USBDAC mode

---

Play a URL

Request format

```
GET /httpapi.asp?command=setPlayerCmd:play:<url>
```

Example response:

```
OK
```

Play Instruction for any valid audio file or stream specified as a `URL`.

Command: `setPlayerCmd:play:<url>`

Parameter	Description
`url`	A complete `URL` for an audio source on the internet or addressable local device `http://89.223.45.5:8000/progressive-flac` example audio file `http://stream.live.vc.bbcmedia.co.uk/bbc_6music` example radio station file

---

Play a M3U File/Playlist

Request format

```
GET /httpapi.asp?command=setPlayerCmd:m3u:play:<url>
```

Example response:

```
OK
```

Play Instruction for any valid `m3u` file or playlist specified as a `URL`. The M3U used extended tags to support coverart URL, title and artist for the tracks.

Command: `setPlayerCmd:m3u:play:<url>`

Parameter	Description
`url`	A complete `URL` for an m3u file source on the internet or addressable local device `http://nwt-stuff.com/Audio/playlists/ProgFLAC.m3u` example audio file `http://nwt-stuff.com/Audio/playlists/bbc_6music.m3u8` example radio station file. The format of `m3u` files is not covered in this documentation. See further information on m3u file formats.

Play Selected Track

Request format:

```
GET /httpapi.asp?command=setPlayerCmd:playindex:<index>
```

Example response:

```
OK
```

The following commands will operate on the selected audio device.

Command: `setPlayerCmd:playindex:<index>`

Param	Description
`index`	play the selected track in current playlist, start from 1, and will play last track when `index` exceed the number of tracks.

ℹ Introduced in version 4.6.328252

Set Shuffle And Repeat Mode

Request format:

```
GET /httpapi.asp?command=setPlayerCmd:loopmode:<mode>
```

Example response:

```
OK
```

Command: `setPlayerCmd:loopmode:<mode>`

Parameter	Description
`mode`	Activates a combination of Shuffle and Repeat modes `0`: Shuffle disabled, Repeat enabled - loop

Parameter	Description
	`1`: Shuffle disabled, Repeat enabled - loop once `2`: Shuffle enabled, Repeat enabled - loop `3`: Shuffle enabled, Repeat disabled `4`: Shuffle disabled, Repeat disabled `5`: Shuffle enabled, Repeat enabled - loop once

---

Control The Playback

Request format

```
GET /httpapi.asp?command=setPlayerCmd:pause
GET /httpapi.asp?command=setPlayerCmd:resume
GET /httpapi.asp?command=setPlayerCmd:onepause
GET /httpapi.asp?command=setPlayerCmd:stop
GET /httpapi.asp?command=setPlayerCmd:prev
GET /httpapi.asp?command=setPlayerCmd:next
```

Example response:

```
OK
```

Control the current playback

Command: `setPlayerCmd:<control>`

Parameter	Description
`pause`	Pause playback
`resume`	Resume playback
`onepause`	Toggle Play/Pause
`stop`	Stop current playback and removes slected source from device
`prev`	Play previous song in playlist
`next`	Play next song in playlist
`pause`	Pause current playback
`resume`	Resume playback from last position, if it is paused

Seeking

Request format:

```
GET /httpapi.asp?command=setPlayerCmd:seek:<position>
```

Example response:

```
OK
```

Seek with seconds for current playback, have no use when playing radio link.

Command: `setPlayerCmd:seek:<position>`

Parameter	Description
`position`	Position to seek to in seconds

Adjusting Volume

Request format:

```
GET /httpapi.asp?command=setPlayerCmd:vol:50
GET /httpapi.asp?command=setPlayerCmd:vol--
GET /httpapi.asp?command=setPlayerCmd:vol%2b%2b
```

Example response:

```
OK
```

Set system volume

Command: `setPlayerCmd:vol<volume>`

Parameter	Description
`volume`	Adjust volume for current device `:vol`: direct volue, value range is `0-100` `--`: Decrease by 6 `%2b%2b`: Increase by 6

Mute and Unmute

Request format:

```
GET /httpapi.asp?command=setPlayerCmd:mute:<mute>
```

Example response:

```
OK
```

Toggle mute for the device

Command: `setPlayerCmd:mute:<mute>`

Parameter	Description
`mute`	Set the mute mode `0`: Not muted `1`: Muted

Play Preset Content

Request format

```
GET /httpapi.asp?command=MCUKeyShortClick:<num_value>
```

Example response:

```
OK
```

Play Instruction for one of the Programmable Presets (maximum 10)

Command: `MCUKeyShortClick`

Parameter	Description
`num_value`	The numeric value of the required Preset Value range is from `0 - 10`

Query Playlist Song Count

Query number of tracks of current playlist, will return the number in plain.

Command: `GetTrackNumber`

Request format:

```
GET /httpapi.asp?command=GetTrackNumber
```

Example response:

```
0
```

Introduced in version 4.6.328252

Play Notification Sound

When this API is used, the device will lower current volume of playback (NETWORK or USB mode only), and play the url for notification sound. Normally used in condition for a door bell in home automation system.

Command: `playPromptUrl:<url>`

Request format:

```
GET /httpapi.asp?command=playPromptUrl:<url>
```

Parameter	Description
`url`	A complete `URL` for an notification audio on the internet or addressable local device

Example response:

```
OK
```

Only works in NETWORK or USB playback mode
Introduced in version 4.6.415145

USB stick/disk playback

Get content list from USB stick/disk

Request format:

```
GET /httpapi.asp?command=getLocalPlayList
```

Example response when a stick/disk is connected and files where found:

```
{
   "num":"3",
   "locallist":[
      {
         "file":"2F6D656469612F736461312F52656164696E672032303136202D20466F616C732E6D7033"
      },
      {
         "file":"2F6D656469612F736461312F52656164696E672032303136202D20524843502E6D7033"
      },
      {
         "file":"2F6D656469612F736461312F52656164696E672032303136202D2042696666792043436C79726F2E6D7033"
      }
   ]
}
```

Example response when no stick/disk is connected or no files where found:

```
no music file
```

Returns a list of music files on an attached USB stick or hard drive (connected with a USB Adaptor)

Command: `getLocalPlayList`

Key	Value-Description
`num`	Number of valid audio files. Value range is from `1-124`.
`locallist`	The array containing the filenames
`file`	A single string of path, filename & file extension. Note the string returned is a `[hexed string]`.

The treatment of hexed data is covered in a seperate section.

Play from USB stick/disk

Request format:

```
GET /httpapi.asp?command=setPlayerCmd:playLocalList:<num_file_index>
```

Example response:

```
OK
```

Based on the return value of `getLocalPlayList` this command can be used to selectively play individual tracks on a connected USB stick/hard disk. The parameter specifies the position of the track within the array returned by `getLocalPlayList`.

Sub-command: `playLocalList`

Parameter	Description
`num_file_index`	The numerical position of an audio file in the USB file index. Value range is from `1-124`.

Track Info from USB stick/disk

Request format:

```
GET /httpapi.asp?command=getFileInfo:<num_file_index_start>:<num_range>
```

Example response for `range=1`:

```
{
   "filename":"2F6D656469612F736461312F52656164696E672032303136202D20466F616C732E6D7033",<br>
   "totlen":"00:00:00",<br>
   "Title":"52656164696E672032303136202D20466F616C732E6D7033",<br>
   "Artist":"756E6B6E6F776E",<br>
   "Album":"756E6B6E6F776E"
}
```

Example response for `range=2`:

```
{
    "num": "2",
    "infolist": [
        {
         "filename":"2F6D656469612F736461312F52656164696E672032303136202D20466F616C732E6D7033",
         "totlen":"00:00:00",
         "Title":"52656164696E672032303136202D20466F616C732E6D7033",
         "Artist":"756E6B6E6F776E",
         "Album":"756E6B6E6F776E"},
       {
         "filename":"2F6D656469612F736461312F52656164696E672032303136202D20466F616C732E6D7033",
         "totlen":"00:00:00",
         "Title":"52656164696E672032303136202D20466F616C732E6D7033",
         "Artist":"756E6B6E6F776E",
         "Album":"756E6B6E6F776E"},
       }
    ]
}
```


Allows to query file information on a connected USB stick or hard drive. Two parameters are passed. The first, the `index` value, stands for "the number of files in the file system" from which the output should start. The second parameter `range` specifies how many files should be analyzed, starting with `index`. So, if `range` is larger than `1`, the response will give informations for multiple music files.

Command: `getFileInfo`

PARAMETER DESCRIPTION

Parameter	Description
`num_file_index_start`	Start Position - the numerical position of an audio file in the USB file index. Value range is from `1-1024`.
`num_range`	The quantity of files to be analysed, starting from the `num_file_index_start` value

RESPONSE DESCRIPTION

Key	Value-Description
`filename`	The filename as `[hexed string]`
`totlen`	Total playing time of that track (in ms)
`Title`	The title of that track. If the info is available, the value is returned as `[hexed string]`, otherwise `UNKNOWN` in plaintext
`Artist`	The artist of that track. If the info is available, the value is returned as `[hexed string]`, otherwise `UNKNOWN` in plaintext
`Album`	The album where this track is from. If the info is available, the value is returned as `[hexed string]`, otherwise `UNKNOWN` in plaintext

The treatment of hexed data is covered in a seperate section.

Multiroom / Multizone

Multiroom is a very special feature in LinkPlay modules. Several devices can be grouped together to form a so-called listening zone, so that all devices in this group play music from one and the same source. The volume can be controlled per device or for the entire group.

If you have already grouped several devices into one zone, then one device of this group is always the host. All other devices are child to this one. This is important to know if you want to play music via Spotify Connect. In the Spotify app, only the host appears in the list of Connect devices! All other devices are hidden.

Add Guest Device to Multiroom Group (Router Mode)

This command will add/join a Guest Device to the Host Device or an existing Multiroom Group (which is assigned to the host device). The command is sent to the Guest Device and the IP Address of Host Device needs to be added to the command as shown below.

Command: `ConnectMasterAp:JoinGroupMaster`:

Request format:

```
GET /httpapi.asp?command=ConnectMasterAp:JoinGroupMaster:eth<ip_address>:wifi0.0.0.0
```

Example Response:

```
OK
```

PARAMETER DESCRIPTION

Parameter	Description
`ip_address`	The IP address of the host device to be removed from the group

The response isn't in JSON format. It is just plaintext.

Multiroom Overview

Command: `multiroom`

The command `multiroom` is the main command, it needs a sub command to execute an action. See the list of current available sub-commands.

Sub-Command	Description
`getSlaveList`	Requests a list of Guest Devices in the Multiroom Group
`SlaveVolume`	Adjusts the volume of a Guest Device
`SlaveKickout`	Removes a Guest Device from the Multi-Room Group
`Ungroup`	disables Multiroom mode on the Host Device, and will split up the entire group

---

Request a list of Guest Devices in a Multiroom Group

Request format:

```
GET /httpapi.asp?command=multiroom:getSlaveList
```

Example response:


```
{
   "slaves": 1,
   "wmrm_version": "4.2",
   "slave_list": [
      {
            "name": "Wohnzimmer",
            "uuid": "FF31F09EFFF1D2BB4FDE2B3F",
            "ip": "10.213.69.106",
            "version": "4.2",
            "type": "WiiMu-A31",
            "channel": 0,
            "volume": 63,
            "mute": 0,
            "battery_percent": 0,
            "battery_charging": 0
        }
   ]
}
```


Example response when there are no Guest Devices connected e.g. This Device is in standalone mode:


```
{
   "slaves": 0,
   "wmrm_version": "4.2"
}
```


Sub-Command: `getSlaveList`

This sub-command requests a list of Guest Devices in the Multiroom Group. A `JSON` table with information about the Guest Devices is returned.

RESPONSE DESCRIPTION (JSON TABLE)

Key	Value-Description
`slaves`	The number of Guest Devices connected to this Host Device. Numeric value.
`wmrm_version`	*!! DOCUMENTATION IN PROGRESS Windows Media Rights Management ? !!*
`slave_list`	Identifier for the array of Guest Devices.

Guest Device Information

Key	Value-Description
`name`	The name of the Guest Device
`uuid`	UUI of the Guest Device
`ip`	Guest Device's IP address
`version`	*!! DOCUMENTATION IN PROGRESS !!*
`type`	Audio Module type `WiiMu-A31`: LinkPlay A31 (used for A50, PRO, MINI, S50+ AMP)
`channel`	Active audio channel `0`: Stereo

Key	Value-Description
	`1`: Left channel `2`: Right channel
`volume`	Current volume. Value range is from `0 - 100`. So can be considered a linear percentage (0% to 100%)
`mute`	Mute status: `0`: Guest Device is unmuted `1`: Guest Device is muted
`battery_percent`	Battery level (if battery is present). Value ranges from `0 - 100`. So can be considered a linear percentage (0% to 100%)
`battery_charging`	Flag that indicates whether the battery is currently charging or not. `0`: Battery not charging `1`: Battery is charging

Mute a Guest Device

Request format:

```
GET /httpapi.asp?command=multiroom:SlaveMute:<ip_address>:<flag_mute>
```

Example Response:

```
OK
```

Mute a specific Guest Device of a Multiroom group.

Sub-Command: `SlaveMute`

PARAMETER DESCRIPTION

Parameter	Description
`ip_address`	The IP address of the Guest Device to control with this command
`flag_mute`	The desired mute status `0`: Unmuted `1`: Muted

The response isn't in JSON format. It is just plaintext.

Remove Guest Device from Multi-Room Group

Request format:

```
GET /httpapi.asp?command=multiroom:SlaveKickout:<ip_address>
```

Example Response:

```
OK
```

Sub-Command: `SlaveKickout`

This command will remove the required Guest Device from the Multi-Room Group

PARAMETER DESCRIPTION

Parameter	Description
`ip_address`	The IP address of the Guest Device to control with this command

ℹ The response isn't in JSON format. It is just plaintext.

Disable Multiroom Mode

Request format:

```
GET /httpapi.asp?command=multiroom:Ungroup
```

Example Response:

```
OK
```

This sub-command disables Multiroom mode on the Host Device, and will split up the entire group. All devices are returned to stand alone mode.

Sub-Command: `Ungroup`

ℹ The response isn't in JSON format. It is just plaintext.

Appendix

Extended M3U Tags

Example M3U file with extended tags and relative URL

```
#EXTM3U
#EXTURL:http://192.168.0.2/test/cover.jpg
#EXTINF:1,P!nk - Just give me a reason
Just Give Me A Reason.mp3
```

Example M3U file with extended tags and absolute URL

```
#EXTM3U
#EXTURL:http://192.168.0.2/test/cover.jpg
#EXTINF:1,P!nk - Just give me a reason
http://192.168.0.2/test/Just%20Give%20Me%20A%02Reason.mp3
```

Standard M3U does not define tags for coverart and title, artist. There's a extended tag to append an image data for coverart, but it's not suitable for streaming device. So we defined a private tag `EXTURL` for the URL of covert. And also referenced the `EXTINF` tag to include the title and artist. With the extra meta data, the playing state on 4STREAM APP would be more friendly.

Tag	Description
`EXTURL`	The URL will be appended after the tag and `:`. It will be used as coverart of the track, and need to be absolute URL.
`EXTINF`	Information will be appended after the tag and `:`. It is in format `N,ARTIST - TITLE` `N` is track index but it's not used. `ARTIST` and `TITLE` is seperated by `-`, better to remove the `-` in title or artist.

Hexed Values

Convert a hexadecimal string representation to a human readable ASCII string.

```
int hex_to_ascii(
    const char *pSrc,
    unsigned char *pDst,
    unsigned int nSrcLength,
    unsigned int nDstLength)
{
    memset(pDst, 0, nDstLength);

    int i, j = 0;
    for (i = 0; i<nSrcLength; i+=2 ) {
        char val1 = pSrc[i];
        char val2 = pSrc[i+1];

        if ( val1 > 0x60)
             val1 -= 0x57;
        else if (val1 > 0x40)
             val1 -= 0x37;
        else
            val1 -= 0x30;

        if ( val2 > 0x60)
            val2 -= 0x57;
        else if (val2 > 0x40)
            val2 -= 0x37;
        else
            val2 -= 0x30;

        if (val1 > 15 || val2 > 15 || val1 < 0 || val2 < 0)
            return 0;

        pDst[j] = val1*16 + val2;
        j++;
    }

    return j;
}
```

Convert a human readable ASCII string to a hexadecimal string representation.

```
int ascii_to_hex(char* ascii_in, char* hex_out, int ascii_len, int hex_len) {
    const char hex[16] = {'0', '1', '2', '3', '4', '5', '6', '7', '8','9', 'A', 'B', 'C', 'D', 'E', 'F'};
    int i = 0;
    int ret = 0;

    memset(hex_out, 0, hex_len);

    while (i < ascii_len) {
        int b = ascii_in[i] & 0x000000ff;
        hex_out[i * 2] = hex[b / 16];
        hex_out[i * 2 + 1] = hex[b % 16];

        ++i;
        ret += 2;
    }

    return ret;
}
```

As you may have noticed in some examples, there are certain values that need to be converted from a string to a hexadecimal value before sending to the API, or from a hexadecimal value to a human readable ASCII string after receiving a response. These values are marked with `[hexed string]`.

Here are two C functions, one to convert a hexadecimal string representation in a human readable ASCII string, and one function to do the opposide:

Changelog

V1.0.2

- Added USBDAC mode for `setPlayerCmd:switchmode`
- Added command `reboot`
- Added command `playPromptUrl` to play link for notification sound
- Updated command `setPlayerCmd:vol` to increase or decrease volume

V1.0.1

- Removed not available command `equalizer`,`getEqualizer`,`setPlayerCmd:hex_playlist`
- Added redirect OTA server API `SetUpdateServer`
- Correct command `setPlayerCmd:switchmode`
- Updated description for some APIs and fixed some format issues

V1.0.0

- Initial release